

Form Fill

Architecture summary — 2-day hackathon build

CORE STACK

Framework	Next.js (React) One project for both UI and the one API route we need — no separate backend project.
Server / backend	Not needed — except one API route All logic runs client-side; only the AI key call needs a server route.
Database	None No accounts or multi-user data to persist for this build.
Storage	IndexedDB (browser) Local-first draft saving so users can pause and resume without a server.
Hosting	Vercel Free tier, deploys Next.js pages and API routes together in one push.

PDF HANDLING

Read / classify	pdf.js + pdf-lib Detects AcroForm vs scanned PDF client-side, no upload needed.
Write-back	pdf-lib Fills real form fields, or overlays text on scanned pages.

ACCESSIBILITY

UI pattern	Sequential one-question wizard Works correctly with any screen reader, no custom navigation needed.
Markup	Semantic HTML + ARIA Ensures compatibility with NVDA, JAWS, VoiceOver out of the box.

OPEN DECISIONS — CONFLICTING APPROACHES

Either option per row is valid — pick one, mix, or add another provider. Fill in the last column with the chosen approach.

Feature	Options	Recommended approach
Form understanding / extraction method	SUGGESTIONS ? Send page image to a multimodal LLM, Extract text/geometry on-device + redact PII + send text only	
AI provider (extraction)	SUGGESTIONS ? Gemini 3 Pro, Claude, GPT-5, Qwen-VL	
Speech-to-text	SUGGESTIONS ?, Groq Whisper, Deepgram Nova-3, AssemblyAI, OpenAI Whisper API, OS-native dictation, On-device Whisper.cpp	

Text-to-speech	SUGGESTIONS ? Browser SpeechSynthesis, ElevenLabs, Azure Neural TTS, Google Cloud TTS, Amazon Polly	
Source file storage	SUGGESTIONS ? Autosaved to IndexedDB by default, Memory-only by default (opt-in to persist)	
API key handling	SUGGESTIONS ? Single serverless proxy route (/api/extract, /api/transcribe), Referrer-restricted key called directly from browser (hackathon only)	

THINGS TO BE FIXED / WATCH FOR

OCR accuracy on messy scans — Handwritten or skewed scans may confuse extraction — needs a fallback/confirmation step.

Safari/Firefox speech quirks — Browser SpeechSynthesis voice quality varies a lot by OS/browser — test on all three early. OR for Hackathon, use chrome only.

IndexedDB storage limits — Large scanned PDFs stored as blobs could hit browser storage quotas on some devices. Delete form after downloading ? Completed form not send to any AI or server- hoping that no other privacy measures needed.

Field-to-coordinate mapping — Overlaying answers on scanned pages needs precise bounding boxes — small offsets look wrong. Are we implementing this now for hackathon ?- writeback on scanned pdf ?

API rate limits mid-demo — Free tiers can throttle unexpectedly — have fallback provider keys ready. Do we need to consider this?

No auth = no cross-device resume — Local-first means a paused form can't be continued on a different device.

QUESTIONS

1. How do we decide which tool reads the form — the PDF library or AI? Is this only based on a confidence score, and we only call AI when confidence is low? Also, when we do call AI, do we send the whole form, or only the text (plus what else)?
2. For the AI calls (reading forms, and converting speech to text), do you suggest we use an API from a big tech company (like Google, Anthropic, or Groq), or should we run a local open-weight model instead, for this hackathon? Also, do we need to worry about token usage or rate limits, or will it be a small enough amount that we don't need to think about it much?